PROJECT PORTFOLIO · 2026

Omar Sherif Elghamry

Backend & AI Engineer. I build production backends with Node.js / NestJS and Python / FastAPI: payment integrations, e-commerce flows, event-driven services and AI features on LangChain and AWS Bedrock.

github.com/omaaarsh · linkedin.com/in/omar-elghamry-3a7256248 · omarsherifelghamry@gmail.com · upwork.com/freelancers/~01aa2d6a5546d432f4

CONTENTS

01 · StyleHub Social Service · graduation project	NestJS · RabbitMQ · Socket.IO
02 · Multi-provider Payments · Techkhana	NestJS · Tap · PayFort
03 · Agentic AI Microservice · Techkhana	FastAPI · LangGraph · Bedrock
04 · Payment Patterns Lab · open source	NestJS 12 · PostgreSQL
05 · Redis Cache Lab · open source	NestJS · Redis · PostgreSQL
06 · Wearly E-Commerce API · open source	ASP.NET Core · EF Core

ABOUT

Who I am

I'm a backend and AI engineer at Techkhana, working on live production systems for a Saudi-market super-app. My day-to-day covers payments, e-commerce checkout, event-driven synchronization, admin authorization and AI integrations, plus the infrastructure they run on. I hold a B.Sc. in Computer Science from Misr University for Science and Technology (CGPA 3.8 / 4.0), where my graduation project, StyleHub, was graded A*.

Experience · Techkhana, 2026 – present

- **Payments:** provider abstraction (factory + strategy), Tap Payments + PayFort, charge / save-card / authorize flows
- **E-commerce checkout:** catalog price snapshot at checkout and a backend-to-backend order flow
- **Event-driven:** Apache Kafka + Kafka Connect / CDC synchronization
- **Admin RBAC:** roles, permissions and guard-based authorization
- **Image processing:** magic-byte validation, WebP conversion and SHA-256 de-duplication over S3
- **AI:** an agentic AI microservice (FastAPI, LangGraph, RAG) and AWS Rekognition with feature flags and usage limits
- **Infrastructure:** Kubernetes (EKS), Docker, GitHub Actions and production support

Skills

AREA	TOOLS
Languages	TypeScript, Python, C#, JavaScript, SQL
Backend	NestJS, Node.js, Express, FastAPI, ASP.NET Core, REST, Swagger / OpenAPI
Data	PostgreSQL, Redis, SQL Server, TypeORM, Entity Framework Core
Messaging & real-time	Apache Kafka, Kafka Connect / CDC, RabbitMQ, Socket.IO
AI	LangChain, LangGraph, RAG, AWS Bedrock, OpenAI API, NumPy
DevOps	Docker, Docker Compose, Kubernetes (EKS), AWS, GitHub Actions, Heroku
Testing	Jest, Supertest, Testcontainers

01 · GRADUATION PROJECT · GRADE A*

StyleHub Social Service

NestJS 11 | TypeScript | PostgreSQL · TypeORM | RabbitMQ | Socket.IO | JWT + RBAC | Docker | GitHub Actions | Heroku | Flutter

ROLE	Sole backend engineer of the social service; also built the Flutter mobile app and a React chat web client
CONTEXT	StyleHub graduation project, Misr University for Science and Technology
TIMELINE	February – September 2026
CODE	github.com/StyleHub1/Social-Media-Service · github.com/omaaarsh/stylehub-chat-frontend

59 REST endpoints

14 feature modules

13 TypeORM migrations

96 / 96 commits by me

Overview

StyleHub is a microservices platform for local fashion businesses that combines e-commerce, social media and AI-driven recommendations. The Social-Media-Service is its social layer: users and brands create posts with photos and videos, follow each other, like and comment, get a personal feed, search for accounts, chat in real time and receive notifications. Every profile and post change is also published to the rest of the StyleHub platform over RabbitMQ, so the e-commerce and AI services stay in sync without calling this service directly.

How the system works

- **One NestJS service, 14 modules:** auth, user, brand, posts, feed, follow, interactions, chat, messaging, notifications, realtime, search, cloudinary and common. Each module is layered controller → service → repository → entity, with DTOs.
- **Request path:** client → HTTP (CORS allow-list) → global validation pipe that strips unknown fields → global guards for the JWT access token, roles and rate limiting (60 requests / 60 s) → controller → service → TypeORM → PostgreSQL.
- **Auth:** JWT access and refresh tokens, with refresh tokens stored in their own table; email verification and password reset tokens; bcrypt hashing; role checks on the posts, interactions and chat controllers. Environment config is validated with Joi at boot.
- **Internal events:** services emit domain events such as `post.created`, `follow.followed` and `auth.user.registered`. Listeners in the feed, auth and notifications modules react asynchronously, for example fanning a new post out to followers' feeds or sending verification emails.
- **Cross-service events:** an event bridge maps internal events to RabbitMQ routing keys (`social.user.*`, `social.brand.*`, `social.post.*`) on a durable topic exchange. If the broker is down, messages are

buffered in memory and flushed on reconnect.

- **Real-time:** a Socket.IO gateway authenticates every socket with the JWT and joins it to a per-user room. It handles `chat:send`, `chat:typing` and `chat:seen`, and messages move through SENT → DELIVERED → SEEN.
- **Background job:** a nightly cron (03:00 UTC) removes feed items older than 30 days.
- **Integrations:** Cloudinary for streamed media uploads and Brevo for transactional email.

Quality and delivery

- 13 TypeORM migrations for safe schema changes over time.
- Jest unit tests plus 12 end-to-end suites that run against a real PostgreSQL through Testcontainers; CI runs them on every push and pull request.
- Multi-stage Docker image; GitHub Actions builds it, pushes it to the Heroku Container Registry and releases it. Docker Compose runs PostgreSQL and RabbitMQ locally with health checks.
- Swagger / OpenAPI docs with a script that syncs the spec to Postman.

Clients I built on top of it

Flutter mobile app: the end-to-end mobile client for the platform. **stylehub-chat-frontend:** a React 19 + TypeScript web client for chat, using TanStack Query and Zustand. It refreshes expired tokens silently with one shared refresh call and a queue for waiting requests, sends messages optimistically with a REST fallback when the socket drops, and loads history with cursor pagination.

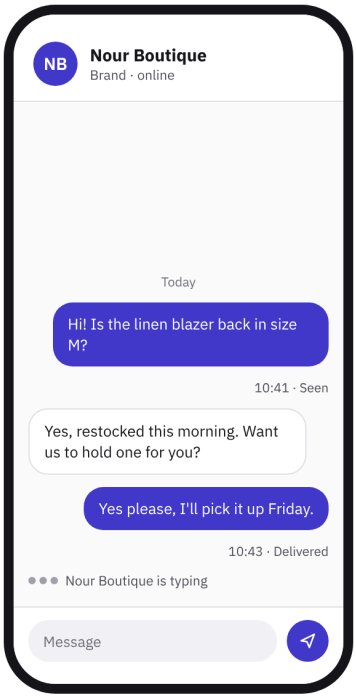
Platform AI

The StyleHub recommendation engine reached an AUC-ROC of 0.93 and 92.87% accuracy. I also built a Python FastAPI AI service for the platform with LangChain, LangGraph and RAG over AWS Bedrock.

GRADUATION PROJECT · GRADE A* · SOLE BACKEND ENGINEER

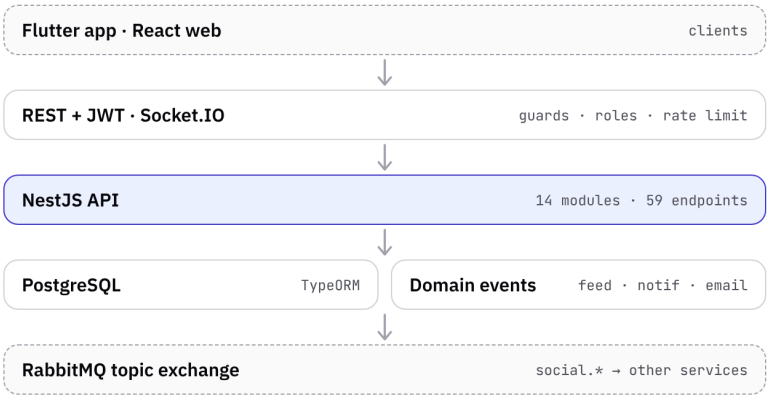
StyleHub Social Service

The social layer of a fashion platform that mixes e-commerce, social media and AI recommendations. Users and brands post, follow, react, chat in real time and get notified, while every change is published to the other StyleHub services.



ILLUSTRATIVE CHAT SCREEN

HOW THE SYSTEM WORKS



59 REST endpoints	**13** TypeORM migrations
12 e2e suites, real Postgres	**96 / 96** commits by me

Fan-out-on-write feed

A new post is pushed to followers' feeds by event listeners; a nightly cron removes items older than 30 days.

Real-time chat

Socket.IO gateway authenticates every socket with the JWT. Typing, seen, and SENT → DELIVERED → SEEN status.

Resilient events

Durable RabbitMQ topic exchange; messages are buffered while the broker is down and flushed on reconnect.

Tested and shipped

Testcontainers e2e in CI, then GitHub Actions → Docker → Heroku Container Registry.

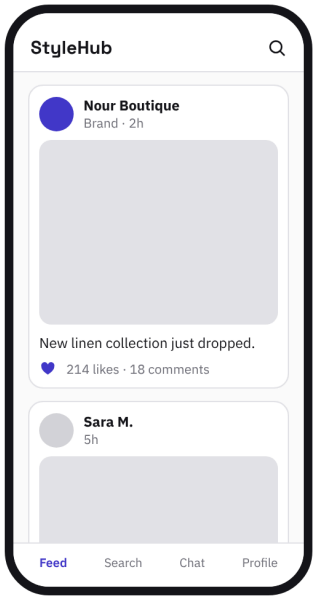
`NestJS` `PostgreSQL` `RabbitMQ` `Socket.IO` `Docker` `Flutter`

Omar Sherif Elghamry
github.com/StyleHub1/Social-Media-Service

STYLEHUB · APP SCREENS POWERED BY THE SOCIAL SERVICE

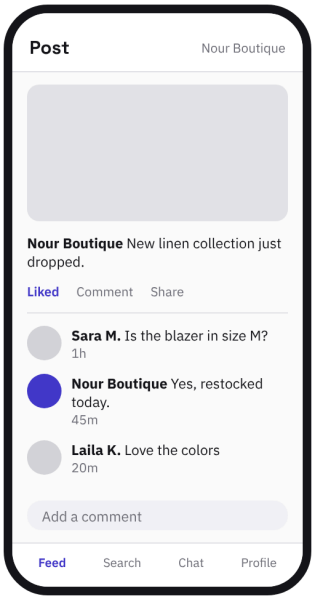
One backend, every social screen

Each screen below is served by a module of the NestJS Social-Media-Service: feed, posts, interactions, follow, notifications, chat and search. Illustrative designs.



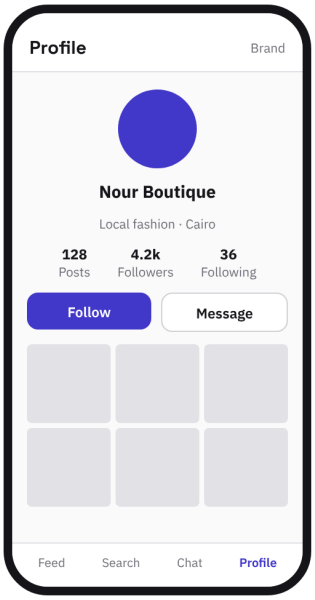
Personal feed

Fan-out-on-write: followed accounts' posts arrive in your feed.



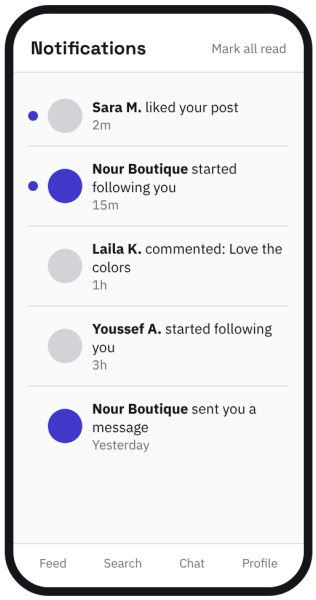
Post and comments

Interactions module: likes and comments, with notifications to the author.



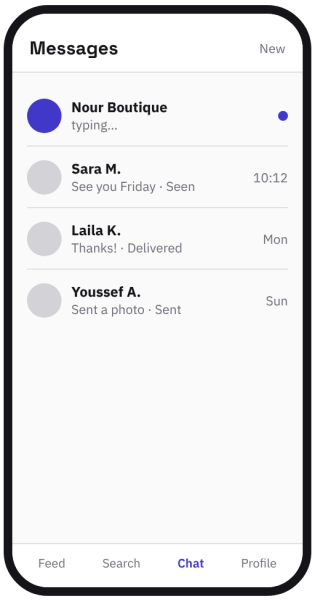
Brand profile

Brand accounts, follow graph and follower counts.



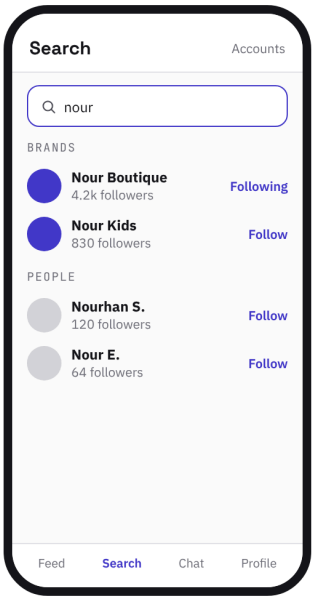
Notifications

Domain events become in-app notifications, pushed live over Socket.IO.



Conversations

Live typing and SENT → DELIVERED → SEEN status per message.



Search

Find users and brands, then follow or open their profile.

02 · TECHKHANA · PRODUCTION

Multi-provider Payment Integration

NestJS | TypeScript | PostgreSQL | Tap Payments | PayFort (Amazon Payment Services)

ROLE	Backend engineer
CONTEXT	Payment architecture for a live Saudi-market super-app at Techkhana
TIMELINE	2026
CODE	Private (company code). The same design is shown publicly in project 04, the Payment Patterns Lab.

Overview

The super-app took payments through PayFort (Amazon Payment Services) and needed to support more than one payment provider. Adding a second provider directly would have spread provider-specific checks through checkout and order code. I refactored payments so every provider sits behind one interface and then integrated Tap Payments next to PayFort.

The problem

Each provider has its own API, request format, statuses and error codes. Without an abstraction, every new provider means touching checkout, orders and payment code, and every flow (charge, save card, authorize) has to be written once per provider and per platform.

What I built

- **Provider abstraction:** payments refactored around a factory + strategy design, so the payment service depends on one provider interface and the factory supplies the right implementation.
- **Tap Payments integration** alongside the existing PayFort integration.
- **Payment flows:** charge, save-card and authorize, using card tokenization.
- **Platform-specific configuration** for mobile and web clients.
- **Clean separation of Checkout → Order → Payment:** checkout stores a price snapshot, so the order keeps the price the customer saw even if catalog prices change later.

How a payment flows

#	STEP	WHAT HAPPENS
1	Client	The mobile or web app starts checkout with its platform-specific configuration.
2	Checkout	Prices are captured as a snapshot.
3	Order	The order is created from the checkout, separate from payment.

4	Payment service	Calls the provider interface: charge, save card or authorize.
5	Provider factory	Supplies the Tap or PayFort implementation.
6	Provider	Tap Payments or PayFort processes the tokenized card.

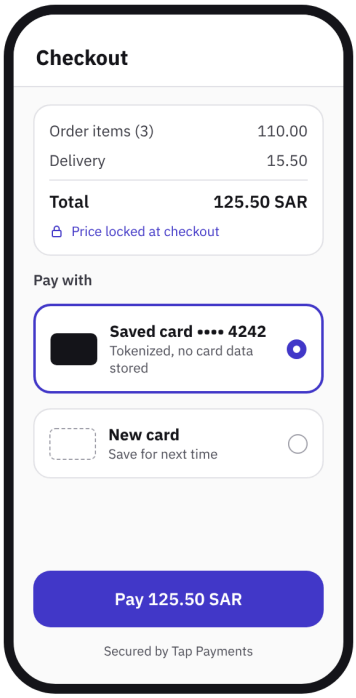
Result

Two payment providers run in production behind one interface. A new provider is a new implementation of the interface, not a change to checkout or orders.

TECHKHANA · BACKEND ENGINEER · LIVE SAUDI-MARKET SUPER-APP

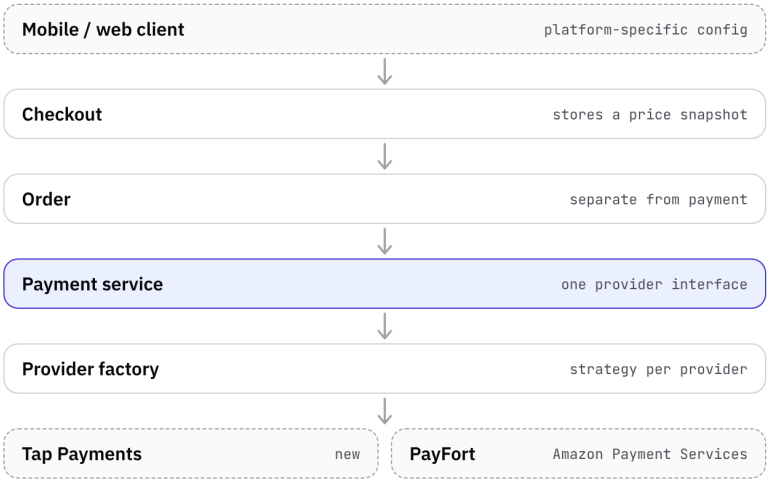
Multi-provider Payments

The app needed more than one payment provider. I refactored payments around one provider interface, so Tap Payments now runs next to the existing PayFort integration and the rest of the code never knows which one is used.



ILLUSTRATIVE CHECKOUT SCREEN

HOW A PAYMENT FLOWS



FLOWS BEHIND THE INTERFACE

Charge	Save card	Authorize
pay now	tokenize a card	hold, capture later

Provider abstraction

Factory + strategy pattern: every provider sits behind one interface, so adding a provider doesn't touch checkout or orders.

Tap next to PayFort

Integrated Tap Payments alongside the existing PayFort (Amazon Payment Services) integration.

Card tokenization

Charge, save-card and authorize flows built on tokenized cards.

Clean boundaries

Checkout → Order → Payment are separate steps, and checkout keeps a price snapshot so later price changes don't affect the order.

NestJS · TypeScript · PostgreSQL · Tap Payments · PayFort

Omar Sherif Elghamry
Backend & AI Engineer

03 · TECHKHANA · PRODUCTION

Agentic AI Microservice

Python FastAPI LangGraph LangChain RAG AWS Bedrock OpenAI API PostgreSQL Docker AWS

ROLE	Primary backend & AI engineer
CONTEXT	Production AI microservice at Techkhana, serving real clients
TIMELINE	2026
CODE	Private (company code)

Overview

I built Techkhana's agentic AI microservice from scratch as its primary engineer. A FastAPI service receives requests and hands them to multi-agent pipelines built with LangGraph. The agents retrieve relevant context with a LangChain RAG system and call large language models through the OpenAI API and AWS Bedrock. Data is kept in PostgreSQL and the service is deployed on AWS.

How the system works

- **API layer:** FastAPI exposes the service to client applications.
- **Orchestration:** LangGraph runs multi-agent pipelines, splitting the work into separate agent nodes that can be tested and improved on their own.
- **Retrieval:** a RAG system built with LangChain grounds answers in the relevant data.
- **Models:** both the OpenAI API and AWS Bedrock are integrated for LLM capabilities.
- **Persistence and deployment:** PostgreSQL for data, Docker, and AWS infrastructure.

What I did

- Designed the full service architecture.
- Integrated OpenAI and AWS Bedrock.
- Built the multi-agent orchestration pipelines with LangGraph.
- Implemented RAG with LangChain.
- Managed data persistence with PostgreSQL and deployed on AWS.
- Resolved real production problems, including performance bottlenecks and model inconsistencies.

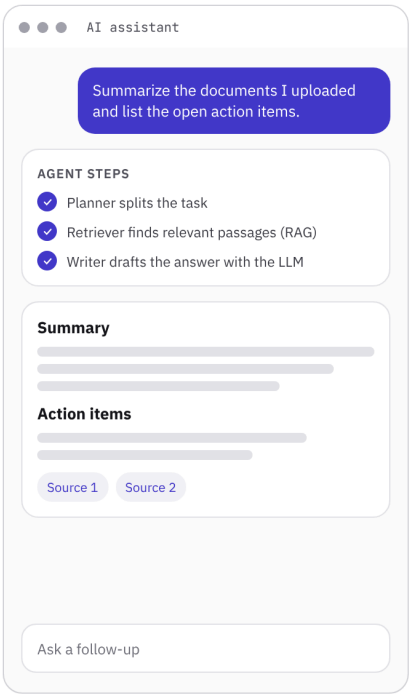
Also at Techkhana

Built RESTful APIs for a construction management platform using Node.js and PostgreSQL, and integrated AWS Rekognition with feature flags and usage limits. This work is backed by an official internship experience letter from Techkhana's CTO.

TECHKHANA · PRIMARY BACKEND & AI ENGINEER · PRODUCTION

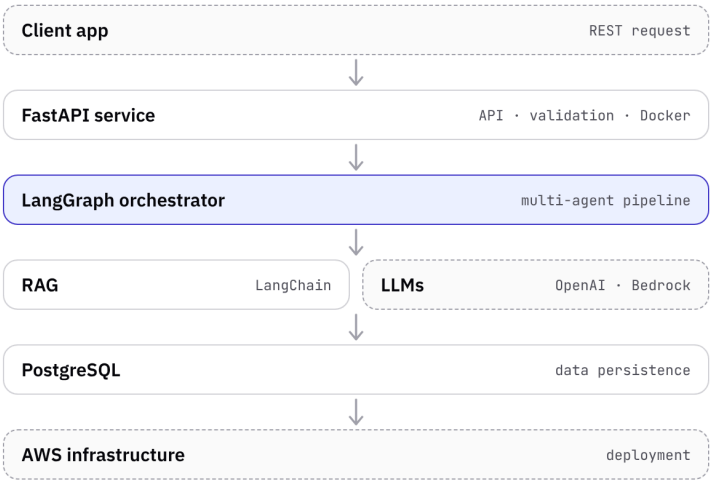
Agentic AI Microservice

An AI service built from scratch that serves real clients. A FastAPI backend hands each request to a team of LangGraph agents that retrieve the right context with RAG and call OpenAI or AWS Bedrock models to answer.



ILLUSTRATIVE CLIENT SCREEN

HOW THE SYSTEM WORKS



WHY AGENTS

LangGraph splits the work into separate agent nodes, so each step (for example retrieval and generation) can be tested, retried and improved on its own instead of living in one giant prompt.

Built from scratch

Designed the full service architecture as the primary engineer, from API to deployment.

Two LLM providers

OpenAI API and AWS Bedrock integrated behind the same service.

Multi-agent + RAG

LangGraph orchestration pipelines and LangChain retrieval for grounded answers.

Production fixes

Resolved real performance bottlenecks and model inconsistencies for live clients.

FastAPI · LangGraph · LangChain · AWS Bedrock · OpenAI · PostgreSQL

Omar Sherif Elghamry
Backend & AI Engineer

04 · OPEN SOURCE · EDUCATIONAL LAB

NestJS Payment Patterns Lab

NestJS 12 | TypeScript 6 strict | PostgreSQL 17 | TypeORM | Swagger | Docker Compose | oxlint

ROLE	Sole author
PUBLISHED	October 2026 · MIT license
CODE	github.com/omaaarsh/nestjs-payment-patterns-lab

Overview

A small, runnable NestJS project that shows how Strategy, Factory, Adapter and Dependency Injection work together in one payment flow: not as four separate examples, but as four roles in the same request. Two simulated providers, PayFort and Tap, speak deliberately different APIs. The application talks to neither directly. It is an educational lab with in-memory mock gateways, no network calls and no real money.

The four patterns

PATTERN	RESPONSIBILITY
Strategy	`IPaymentProvider` is the contract; both adapters implement it; `PaymentsService` holds one and delegates. There is no provider branching in the service.
Factory	One function maps `PAYMENT_PROVIDER` to an adapter. An unsupported value stops the app at startup, with no silent fallback.
Adapter	Each adapter translates the internal request into its gateway's format, and the gateway's responses, statuses and errors back into one normalized result.
Dependency Injection	NestJS binds the factory's result to a token and passes it into `PaymentsService` through its constructor.

What the adapters translate

	PAYFORT (MOCK)	TAP (MOCK)	THE APP SEES
Amount	minor units `"12550"`	decimal `"125.50"`	`"125.50"`
Transaction id	`fort_id`	`chg_…` / `auth_…`	`providerTransactionId`
Success	status `"14"` / `"02"`	`CAPTURED` / `AUTHORIZED`	`CAPTURED` / `AUTHORIZED`
Decline	status `"13"` / `"03"`	`DECLINED`	`FAILED` + `DECLINED`
Outage	503 error	503 error	503, stored as `PROVIDER_UNAVAILABLE`

Request flow

- The controller validates the body: amount is a positive decimal string and currency is an ISO 4217 code.
- The service saves a `PENDING` payment, calls `provider.charge()` on the injected adapter, and stores the normalized result.
- If the provider fails, the payment is marked `FAILED` with a reason before the 502 / 503 reaches the client. A decline is a business result (201 + `FAILED`), not an error.
- Money stays a string end to end (`numeric(12,2)`) to avoid floating-point errors. The schema comes from explicit migrations.

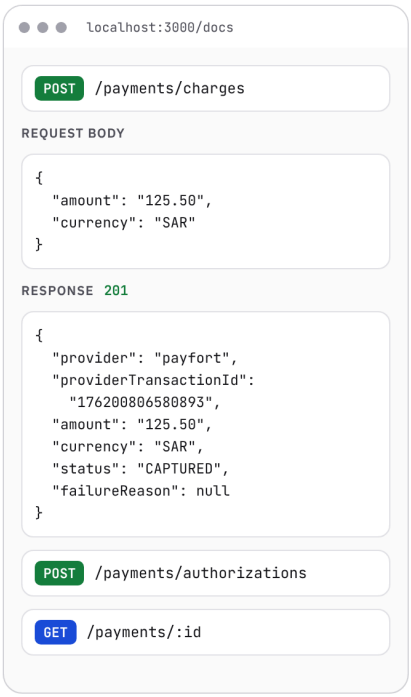
Documented trade-offs

The README is honest about limits: no signatures, 3-D Secure, webhooks or idempotency keys; one provider per process; no capture, void or refund; and a possible stuck `PENDING` payment between two writes, which a real system would reconcile with the provider.

OPEN SOURCE LAB · NESTJS 12 · SIMULATED GATEWAYS

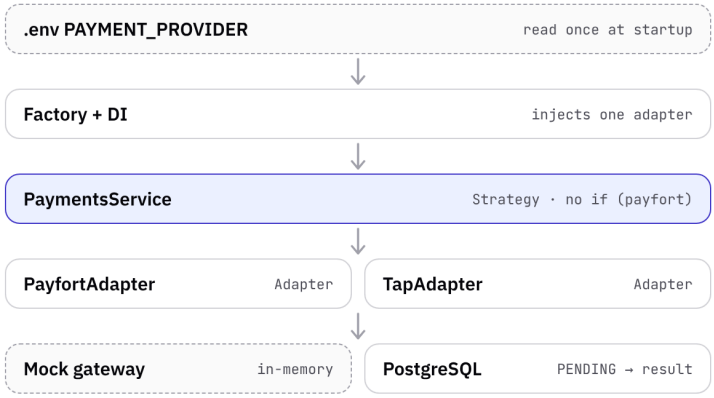
Payment Patterns Lab

Four design patterns, one payment request. PayFort and Tap speak very different APIs; the app talks to neither directly. Switch providers with one environment variable and the service code doesn't change.



SWAGGER UI FROM THE README EXAMPLE

ONE CHARGE REQUEST



WHAT ADAPTERS HIDE	PAYFORT	TAP	APP SEES
Amount	"12550"	"125.50"	"125.50"
Txn id	fort_id	chg_…	providerTxnId
Success	status "14"	CAPTURED	CAPTURED
Outage	503 error	503 error	503 + saved

Strategy

IPaymentProvider is the contract; the service holds one and delegates. No provider branching anywhere.

Factory

The one place that maps PAYMENT_PROVIDER to an adapter. Unknown values stop the app at startup.

Adapter

Translates each gateway's requests, statuses and errors into one normalized result.

Dependency Injection

NestJS binds the factory result to a token and passes it in through constructors.

NestJS 12 · TypeScript strict · PostgreSQL 17 · TypeORM · Docker Compose

Omar Sherif Elghamry
github.com/omaaarsh/nestjs-payment-patterns-lab

05 · OPEN SOURCE · MEASURED LAB

Redis Cache Lab

NestJS 11 | TypeScript | Redis 7 | ioredis | Keyv | cache-manager | PostgreSQL 16 | Docker Compose

ROLE	Sole author
PUBLISHED	September 2026
CODE	github.com/omaaarsh/redis-cache-lab

466 ms → 1.7 ms
repeated GET /products, Postgres vs Redis

50 → 1
database queries for 50 concurrent cold reads

Overview

A hands-on lab that puts a Redis cache in front of a slow PostgreSQL read path in NestJS. It solves the two classic caching problems: serving stale data after writes, and a cache stampede when many requests hit the same cold key at once. The slow query is simulated with a 400 ms delay so the effect of the cache is easy to measure.

How the system works

- **Read path (cache-aside):** `GET /products` → service → Redis. On a hit, the result returns at once; on a miss, the service reads PostgreSQL, writes the result to Redis with a TTL, and returns it. Keys are `products` for the list and `product:{id}` for single items.
- **Write path (event-driven invalidation):** `POST`, `PUT` and `DELETE` save to PostgreSQL and emit `cache.updated`; the cache service listens and deletes the affected list and item keys.
- **Stampede protection (single-flight):** the service keeps the in-flight promise for a cold read, so concurrent callers wait on the same promise and only one database query runs.
- **Cache layer wiring:** ioredis client → Keyv store adapter → cache-manager module → cache service, instead of the default one-line setup.
- **Graceful degradation:** if Redis is unreachable, the client is null and the app falls back to an in-memory cache and keeps serving.

What it demonstrates

Caching that is fast, correct after writes, safe under concurrent load, and resilient when the cache itself is down, with the gains measured instead of assumed.

OPEN SOURCE LAB · NESTJS + REDIS · MEASURED RESULTS

Redis Cache Lab

A slow PostgreSQL read path made fast with Redis, without serving stale data and without flooding the database when a cold key gets many requests at the same moment.

results

GET /PRODUCTS LATENCY

Cold, from Postgres **466 ms**

Warm, from Redis **1.7 ms**

DB QUERIES FOR 50 CONCURRENT COLD READS

Without single-flight **50**

With single-flight **1**

```
$ curl localhost:3005/products
  cache MISS → postgres
$ curl localhost:3005/products
  cache HIT  → redis
$ curl -X PUT …/products/:id
  emit cache.updated
  DEL products, product:{id}
```

RESULTS AS MEASURED IN THE README

READ PATH

GET /products — client

↓

DemoService — cache-aside + single-flight

↓

Hit → Redis 7 — ~1.7 ms

Miss → PostgreSQL — then set TTL

WRITE PATH

POST / PUT / DELETE — client

↓

Save to PostgreSQL — emit cache.updated

↓

CacheService — deletes list + item keys

CACHE LAYER, BUILT BY HAND

`ioredis client → Keyv store → cache-manager → CacheService`

If Redis is down, the client is null and the app falls back to an in-memory cache instead of crashing.

Cache-aside with TTL

Reads check Redis first; on a miss, load from Postgres and store with an expiry.

Event-driven invalidation

Every write emits cache.updated, so stale list and item keys are removed right away.

Stampede protection

Concurrent cold callers share one in-flight promise, so only one database query runs.

Graceful degradation

Falls back to in-memory storage when Redis is unavailable; the API keeps serving.

NestJS 11 · Redis 7 · ioredis · PostgreSQL 16 · Docker Compose

Omar Sherif Elghamry
github.com/omaaarsh/redis-cache-lab

06 · OPEN SOURCE · SOLE AUTHOR

Wearly E-Commerce API

C# | ASP.NET Core (.NET 9) | Entity Framework Core | SQL Server | BCrypt | Swagger | Docker

ROLE	Sole author
TIMELINE	April – May 2025
CODE	github.com/omaaarsh/Wearly-Clothing-ECommerce-back-end-

Overview

Wearly is a clothing store. This repository is its backend: a REST API that powers customer accounts, catalog browsing for men, women and kids, a shopping cart and checkout for a separate web front-end.

Layered architecture

LAYER	PURPOSE
Controllers	Handle HTTP requests under `/api/v1`, call services, return responses.
Services	Business rules and validation; coordinate repository calls.
Repositories	Data access with EF Core over SQL Server.
Models	Customer, Product, Category, Cart, CartItem, Order and OrderItem, plus DTOs.

All services and repositories are registered through interfaces with dependency injection, which keeps the layers loosely coupled and easy to test.

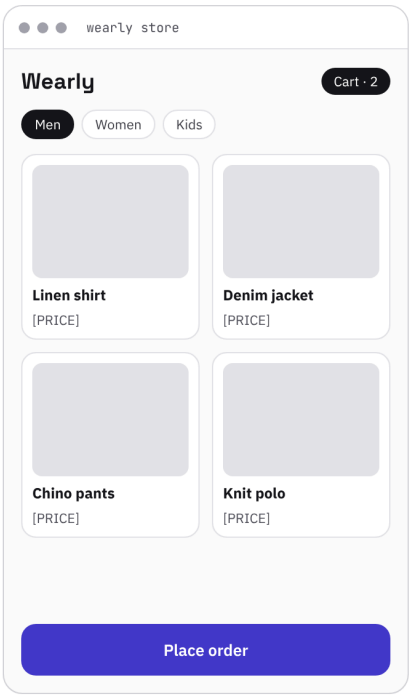
Key flows

- **Accounts:** register, login, logout and auth status. Passwords are hashed with BCrypt; login issues an HttpOnly, SameSite=Lax session cookie with a 30-day sliding expiry. Unauthenticated calls get a JSON 401 instead of a redirect.
- **Catalog:** products, product details, categories and gender-filtered lists.
- **Cart:** authenticated view, add, update, remove and clear.
- **Placing an order:** read the cart → check every product exists and has stock → snapshot the unit price into each order item → compute the total → decrement stock → save the order → clear the cart. Order history is available per customer.
- **API quality:** custom validation attributes (password complexity, phone format), a global exception handler with one JSON error shape, and Swagger UI in development.

OPEN SOURCE · ASP.NET CORE (.NET 9) · SOLE AUTHOR

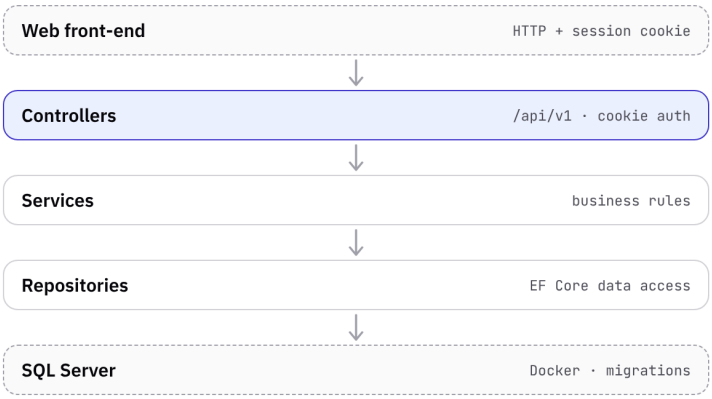
Wearly E-Commerce API

The server side of a clothing store: customer accounts, a catalog for men, women and kids, a shopping cart and checkout, built as a clean layered REST API for a separate web front-end.

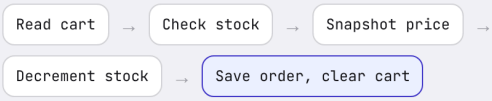


ILLUSTRATIVE STORE SCREEN

LAYERED ARCHITECTURE



PLACING AN ORDER



Secure accounts

BCrypt password hashing and HttpOnly cookie sessions with a 30-day sliding expiry.

Catalog

Products, categories and gender-filtered lists for men, women and kids.

Cart and orders

Authenticated cart (add, update, remove, clear) and order history.

Clean API

Versioned /api/v1, Swagger docs, custom validation and one consistent JSON error format.

C# · ASP.NET Core · EF Core · SQL Server · Swagger

Omar Sherif Elghamry
github.com/omaaarsh/Wearly-Clothing-ECommerce-back-end-

CONTACT

Let’s build something reliable.

Open to freelance backend work: payments, APIs, event-driven services and AI features.

Email	omarsherifelghamry@gmail.com
GitHub	github.com/omaaarsh
LinkedIn	linkedin.com/in/omar-elghamry-3a7256248
Upwork	upwork.com/freelancers/~01aa2d6a5546d432f4